REMOVING TOKENIZATION BIAS: EXACT BYTE-LEVEL PREDICTIONS FROM TOKENIZED LANGUAGE MODELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Tokenization compresses text and makes language models (LMs) tractable, but it also makes a model's predictions depend on how the prompt happens to be segmented. We call the resulting mismatch between a tokenized LM's next-token distribution and the byte-level distribution it implies *tokenization bias*. It is most damaging when a prompt ends in the middle of a token, as in fill-in-the-middle (FIM) code completion, where the correct continuation can receive exactly zero probability. We show how to convert the next-token predictions of a tokenized LM into next-byte predictions at inference time, with no retraining: instead of conditioning on a single greedy tokenization of the prompt, we marginalize over the valid token sequences that cover the prompt, and find them with an efficient backward search. On the HumanEval Random Span infilling benchmark with CodeLlama-7b, byte-level prediction raises the greedy pass rate for the Suffix-Prefix-Middle template from 45.0 to 63.9, ahead of token healing (62.7) and token alignment (63.0), and also gives a small gain for Prefix-Suffix-Middle (64.9 to 65.7). Because next-byte distributions live in a vocabulary-independent space, models with different tokenizers can be ensembled by averaging them: top-2 byte-level ensembles improve over the best single model on MMLU (65.4 vs. 63.4), HumanEval (42.1 vs. 38.4) and other benchmarks, but not when the members differ greatly in ability (GSM8K: 55.8 vs. 61.5).

1 INTRODUCTION

State-of-the-art language models do not read text byte by byte. A tokenizer, typically trained with byte-pair encoding (Gage, 1994; Sennrich et al., 2016) or a related algorithm (Kudo & Richardson, 2018), first compresses the input into a sequence of tokens from a fixed vocabulary, and the model is trained and queried over those tokens (Brown et al., 2020; Touvron et al., 2023). Compression shortens sequences and so reduces computation, but it is a lossy interface between the model and the text it is meant to describe. Several recent studies document consequences of this choice, from uneven costs across languages (Petrov et al., 2023) to degraded arithmetic (Singh & Strouse, 2024) and under-trained tokens (Land & Bartolo, 2024).

In this paper we study a specific, mathematically precise consequence, which we call *tokenization bias*. A deterministic tokenizer maps every byte string to exactly one token sequence. A model trained on such sequences therefore sees only *canonical* segmentations of the past. When a prompt ends exactly where the tokenizer would end a token, this is harmless. When a prompt ends *mid-token*, however, the canonical tokenization of the prompt is not a prefix of the canonical tokenization of the text the user has in mind, and the model is conditioned on a history it would never have produced for the full text. Figure 1 (left) shows a code-completion example: after the prompt `return 5 + a`, the token-level model assigns probability 0.0 to the token `a`, which is the correct continuation (the intended text is `aa`), because under greedy tokenization the byte `a` followed by another `a` would have been merged into a different token. The same mechanism is visible in the simplest possible setting, a first-order Markov chain over two symbols (Figure 4): a tokenized model never samples a transition that the ground-truth chain takes with probability $1 - \alpha$.

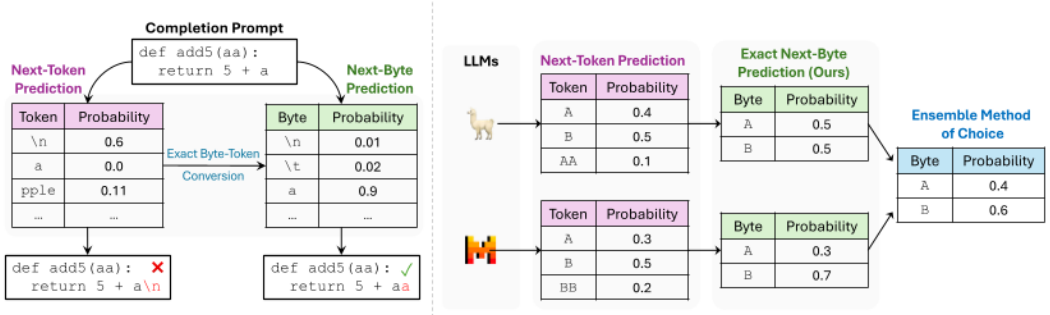


Figure 1: Left: tokenized LMs can experience tokenization bias when prompts end mid-token, as in this code completion example. The correct solution, `a`, has zero probability of being chosen at the token level. Our method avoids this problem and can predict the correct token while using the same model. Right: our method maps next-token predictions of arbitrary tokenized LMs to statistically equivalent next-byte predictions (see Section 3 for details). This enables any model ensemble strategy, such as averaging or mixture of experts.

Practical systems patch this problem heuristically. For code infilling, where the Suffix-Prefix-Middle (SPM) template places the cursor in the middle of a token by construction (Bavarian et al., 2022), *token healing* (Dagan et al., 2024) backs up to a token boundary and constrains generation to tokens that match the removed text, and *token alignment* (Athiwaratkun et al., 2024) matches characters to complete a partial subword. These fixes are specific to one setting and give no guarantee that the resulting distribution is the one the model implies at the byte level. A second limitation is that token-level distributions of models with different vocabularies are over disjoint outcome spaces, so such models cannot simply be averaged.

Our starting hypothesis is that tokenization bias is an artifact of conditioning on one deterministic tokenization of the past, and that a statistically equivalent byte-level process can be recovered from any trained tokenized LM without retraining. The probability of a byte string is the total probability of all (valid) token sequences that decode to it, and the next-byte distribution follows by summing the probabilities of all next tokens that begin with a given byte, conditioned on every valid encoding history. We turn this into an inference-time wrapper around an unmodified model (Figure 1, right).

Contributions.

- **Formalization.** We define tokenization bias and the notions of valid encoding and cover encoding, and state a *Byte-Token Representation Lemma* that expresses the probability of a byte prefix as a sum over cover encodings (Section 3).
- **Algorithm.** We describe an inference-only procedure that samples the next byte exactly from a tokenized LM, using a backward search over the last few bytes to find cover encodings efficiently, so that any tokenized model becomes a byte-level model without weight updates.
- **Robust FIM.** On HumanEval Random Span infilling with CodeLlama-7b, byte-level prediction removes most of the drop incurred by the SPM template (45.0 $\rightarrow$ 63.9 greedy pass rate) and outperforms token healing and token alignment (Section 4.3).
- **Vocabulary-agnostic ensembling.** We average the next-byte distributions of models with different tokenizers, and report where this helps (MMLU, HumanEval, PIQA, NQ, TriviaQA) and where it does not (GSM8K) (Section 4.4).

2 Related Work

Tokenization and its side effects. Subword tokenization became standard through byte-pair encoding for neural models (Sennrich et al., 2016), building on a compression algorithm (Gage, 1994),

and through language-independent implementations such as SentencePiece (Kudo & Richardson, 2018); Mielke et al. (2021) survey open-vocabulary modeling and tokenization. A line of work shows that the segmentation chosen by the tokenizer affects model behavior and fairness (Petrov et al., 2023; Singh & Strouse, 2024; Land & Bartolo, 2024), and a growing body of theory studies tokenizers formally, including BPE (Zouhar et al., 2023), tokenization of Markov data (Rajaraman et al., 2024), and statistical and computational foundations (Gastaldi et al., 2024). Closest in spirit to our marginalization argument, Cao & Rimell (2021) argue that LMs should be evaluated by marginal likelihood over tokenizations rather than over one canonical tokenization. Subword regularization (Kudo, 2018) and BPE-dropout (Provilkov et al., 2020) expose a model to multiple segmentations *during training*. In contrast to these, our method leaves training and the model untouched, and uses the model's own predictions on *canonical* segmentations to compute exact byte-level probabilities at inference time.

Token-free models. An alternative is to remove tokenization altogether and model bytes directly (Xue et al., 2022), with architectures designed to cope with long byte sequences (Yu et al., 2023; Slagle, 2024; Wang et al., 2024). These approaches require training new models. Our method instead converts already-trained tokenized models into byte-level predictors.

Prompt-boundary heuristics and constrained decoding. Token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) address prompts that end mid-token by re-tokenizing the prompt boundary or by matching characters to subwords; they are the baselines we compare with. Related misalignment between a model's tokens and externally imposed character-level constraints arises in constrained generation (Beurer-Kellner et al., 2024). These approaches modify the conditioning text or the set of allowed tokens, whereas we compute the byte-level distribution implied by the model itself.

Fill-in-the-middle. FIM training (Bavarian et al., 2022) lets autoregressive models infill code; the PSM and SPM prompt templates are standard, and infilling models include InCoder (Fried et al., 2023), StarCoder (Li et al., 2023) and Code Llama (Rozière et al., 2023). SPM prompts end with the prefix, which can sit in the middle of a token, making them a natural stress test for tokenization bias.

Ensembling models with different vocabularies. Combining LMs is attractive, but their token distributions are not directly comparable. Existing approaches include ranking and fusing generated outputs (Jiang et al., 2023b), distilling several models into one (Wan et al., 2024), and aligning or projecting between vocabularies (Xu et al., 2024; Yu et al., 2024). Mixture-of-experts architectures (Jiang et al., 2024) combine experts that share a single tokenizer. Mapping every model to the same byte space, as we do, lets simple distribution-level averaging apply with no vocabulary alignment step.

3 METHODOLOGY

3.1 SETTING AND TOKENIZATION BIAS

We write $x_1^n = (x_1, \ldots, x_n)$ for a byte string over the byte alphabet $\mathcal{B}$ and $t_1^k = (t_1, \ldots, t_k)$ for a token sequence over a vocabulary $\mathcal{V}$.

Definition 1 (Tokenizer)**.** *Each token of $\mathcal{V}$ stands for a non-empty byte string, and* decode *maps a token sequence to the concatenation of its tokens' bytes. The tokenizer* encode *is deterministic (e.g. BPE with a fixed merge list): it maps a byte string to a single token sequence, with* $\mathrm{decode}(\mathrm{encode}(x_1^n)) = x_1^n$.

Definition 2 (Tokenized LM)**.** *A tokenized LM is an autoregressive model $P(t_{k+1} \mid t_1^k)$ over $\mathcal{V}$, trained on sequences produced by* encode*. It defines $P(t_1^k) = \prod_{i=1}^{k} P(t_i \mid t_1^{i-1})$.*

Definition 3 (Valid encoding)**.** *A token sequence t_1^k is a* valid encoding *if* $\mathrm{encode}(\mathrm{decode}(t_1^k)) = t_1^k$*; otherwise it is* invalid*.*

A deterministic tokenizer makes many token sequences impossible: with BPE, the sequence `y|u|m|m|y|u` decodes to `yummyu`, but the merge rules would have produced a different seg-

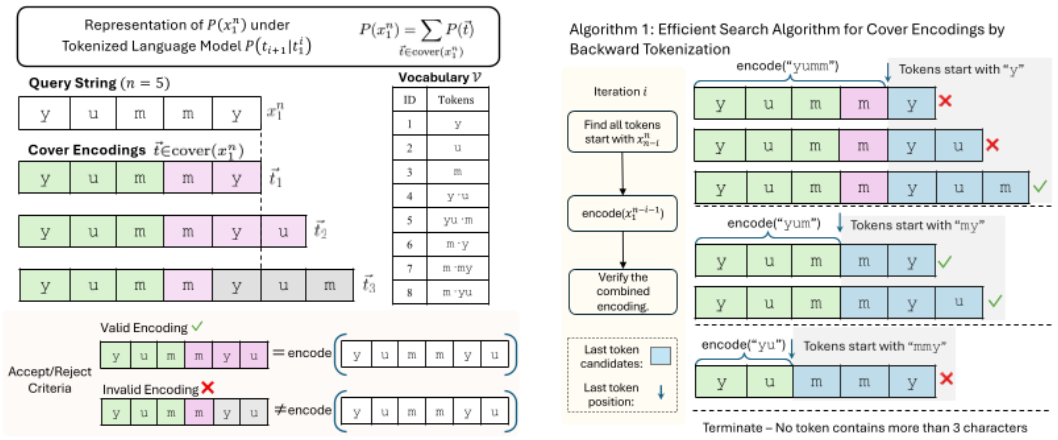


Figure 2: Left: representation of $P(x_1^n)$ using a tokenized LM, with an example of cover encodings and valid/invalid encodings. Right: illustration of Algorithm 1 for the search for cover encodings by backward tokenization. Green ticks and red crosses denote valid and invalid encodings respectively (Definition 3). The termination step can be easily implemented although it is not shown in the algorithm. We use BPE in this example.

mentation of that string, so the sequence is invalid (Figure 2, bottom left). A model trained only on outputs of encode places (ideally) no mass on invalid sequences, and this is what prunes the search space below.

Tokenization bias. The distribution one actually wants after a byte prompt x_1^n is $P(x_{n+1} \mid x_1^n)$, the next-byte distribution of the byte process that the tokenized LM models. The standard recipe instead tokenizes the prompt, $t_1^k = \text{encode}(x_1^n)$, and reads off $P(t_{k+1} \mid t_1^k)$. These two differ in general. In particular, if the true text continues in a way for which encode would have merged the last prompt token(s) with the next byte, then $P(\cdot \mid \text{encode}(x_1^n))$ gives that continuation probability zero (Figures 1 and 4). We call this discrepancy *tokenization bias*.

3.2 Cover encodings and the Byte-Token Representation Lemma

To compute the probability of a byte string x_1^n we need all token sequences that account for it.

Definition 4 (Cover encodings)**.** *The set* $\text{cover}(x_1^n)$ *contains every valid encoding* t_1^k *such that* x_1^n *is a prefix of* $\text{decode}(t_1^k)$ *and* $\text{decode}(t_1^{k-1})$ *is strictly shorter than* n *bytes; that is, the last token is the first one that reaches or passes position* n*.*

The query string yummy of Figure 2 ($n = 5$) has several cover encodings of different lengths: some end exactly at byte n, and others end with a token that extends beyond it. Summing over them gives the byte-level probability.

Lemma 1 (Byte-Token Representation)**.** *Assume the tokenized LM places probability only on valid encodings. Then for every byte string* x_1^n*,*

$$P(x_1^n) \;=\; \sum_{t_1^k \in \text{cover}(x_1^n)} P(t_1^k). \tag{1}$$

Proof sketch. Under the assumption, the events "the (valid) token sequence decodes to a string with prefix x_1^n" and "the sequence has, as its shortest prefix reaching n bytes, a cover encoding t_1^k" coincide, and distinct cover encodings correspond to disjoint events, because the shortest prefix is unique. The probability of each such event is $P(t_1^k)$. Appendix A gives the argument in more detail.

The next-byte distribution is the ratio of two such sums:

$$P(x_{n+1} \mid x_1^n) = \frac{P(x_1^{n+1})}{P(x_1^n)} = \frac{\sum_{t \in \text{cover}(x_1^{n+1})} P(t)}{\sum_{t \in \text{cover}(x_1^n)} P(t)}. \tag{2}$$

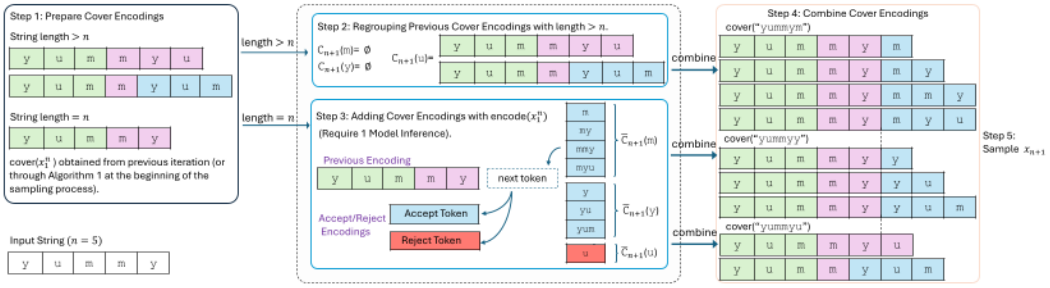


Figure 3: Illustration of the sampling process for x_{n+1} from a tokenized LM $P(t_{k+1} \mid t_1^k)$, with the example following Figure 2. We do not include $P(\vec{t})$ for simplicity. Details in Section 3.4.

Because every term of the numerator comes from a cover encoding of x_1^{n+1}, equivalently from a cover encoding t_1^k of x_1^n plus a next token whose first unread byte is x_{n+1}, the numerator is obtained by *aggregating the probabilities of all next tokens that begin with that byte, conditioned on each valid encoding history*. This is the marginalization that removes the dependence on a single tokenization of the past.

3.3 Reverse search for cover encodings

Enumerating all valid token sequences that decode to x_1^n is infeasible for long contexts. We observe that a cover encoding is determined by its *last* tokens: the tokens that reach byte n start within the last few bytes, since no token is longer than the longest vocabulary entry. We therefore search backwards from the end of the string (Algorithm 1, Figure 2, right). At iteration i (Algorithm 1, Appendix B) we (1) find all vocabulary tokens that start with the suffix x_{n-i}^n, which are candidates for the last token beginning at position $n - i$; (2) encode the remaining prefix, $\mathrm{encode}(x_1^{n-i-1})$, and append the candidate token; and (3) verify that the combined sequence is a valid encoding (Definition 3), keeping it if so.

The search stops when no vocabulary token is long enough to start at the current position and still cover x_n ("no token contains more than 3 characters" in the example of Figure 2). The number of iterations is thus bounded by the maximum token length and not by the context length, and the prefix $\mathrm{encode}(x_1^{n-i-1})$ is a single tokenization that need not be searched.

3.4 Sampling the next byte

Sampling is organized as in Figure 3. We maintain the cover encodings of the current byte string (obtained from the previous iteration, or by the search above at the beginning of the sampling process) together with their probabilities. To sample x_{n+1}:

1. **Regroup.** Cover encodings whose last token extends beyond byte n already determine their next byte; they are regrouped by that byte and contribute to the cover set of x_1^{n+1} for the matching byte only.
2. **Extend.** Cover encodings that end exactly at byte n require one model inference to obtain $P(t_{k+1} \mid t_1^k)$. Each candidate next token is accepted or rejected according to whether the extended sequence is a valid encoding; accepted tokens are grouped by their first byte.
3. **Combine and sample.** The contributions are combined per next byte, normalized as in Eq. equation 2, and x_{n+1} is sampled (or the argmax is taken for greedy decoding). The cover set for x_1^{n+1} then becomes the starting point for the next step.

The procedure needs no weight updates and works as a decoding wrapper around any tokenized LM with a deterministic tokenizer.

3.5 ENSEMBLING IN BYTE SPACE

For models $j = 1, \ldots, M$ with possibly different vocabularies and tokenizers, each model yields its own exact next-byte distribution $P_j(x_{n+1} \mid x_1^n)$ via Eq. equation 2. All of them are distributions over the same byte alphabet, so they can be combined with any rule; we use a simple average,

$$P_{\text{ens}}(x_{n+1} \mid x_1^n) = \frac{1}{M} \sum_{j=1}^{M} P_j(x_{n+1} \mid x_1^n), \tag{3}$$

and sample the next byte from it. Each model then updates its own cover encodings with the chosen byte, so that members never need to agree on a segmentation (Figure 1, right).

4 EXPERIMENTS

We evaluate (i) whether byte-level conversion removes tokenization bias on synthetic Markov data, (ii) whether it helps code infilling when prompts end mid-token, (iii) whether it preserves the quality of the underlying models on standard benchmarks, and (iv) whether averaging in byte space helps. All numbers are copied from our experimental log; we report one value per cell and no confidence intervals.

4.1 SETUP

Models. We use general-purpose models Llama2-7B (Touvron et al., 2023), Yi-1.5-6B (Young et al., 2024) and Mistral-7B-v0.3 (Jiang et al., 2023a), and code models CodeLlama-7b, CodeLlama-13b (Rozière et al., 2023) and Yi-Coder-1.5B. All are Transformer LLMs. For the synthetic experiments we use first- and third-order Markov chains and a 6-layer GPT-2-style Transformer (Radford et al., 2019) trained on synthetic data.

Tasks and metrics. Synthetic: recovery of the chain's transition probabilities. Code infilling: the HumanEval Random Span benchmark (Bavarian et al., 2022) for Python, with both the PSM and SPM prompt templates, reported as greedy pass rate and pass rate @10 (the maximum over a temperature sweep). Knowledge and reasoning: MMLU (Hendrycks et al., 2021), PIQA (Bisk et al., 2020), NQ (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017) and GSM8K (Cobbe et al., 2021), reported as accuracy, comparing token-level to byte-level probabilities. Code generation: HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), pass@1.

Baselines. Standard token-level prediction (default autoregressive generation); token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) for infilling (applicable to SPM, so no PSM entries); and, for ensembles, a top-2 voting strategy and the individual members. Our method ("Byte-level") applies the conversion of Section 3 to the same model weights; the ensemble ("Top-2 Ensemble") averages the byte-level distributions of the top-2 performing models (Section 3.5). We also ablate beam search width 1, 2, 4 for byte-level FIM.

4.2 TOKENIZATION BIAS ON SYNTHETIC DATA

In a first-order Markov chain over bytes `A` and `B`, with a vocabulary containing `A`, `B` and the merged token `AA`, tokenizing with BPE forces `A` to be followed by `B` whenever the single token `A` appears in the history (Figure 4). The probability of the next byte being `A` given the context token `A` therefore becomes 0.0 in token space, whereas the ground truth is $1 - \alpha$.

We then trained a 6-layer GPT-2-style model on data from a third-order Markov chain. Figure 5 compares, for each possible context of the last three bytes, the ground-truth probability that the next byte is `A`, the biased estimate obtained by conditioning on the tokenized prompt, and the estimate obtained with our byte-token conversion. The trained tokenized model exhibits severe bias in its predictive distributions, whereas the converted predictions closely follow the ground truth, which our method thereby recovers.

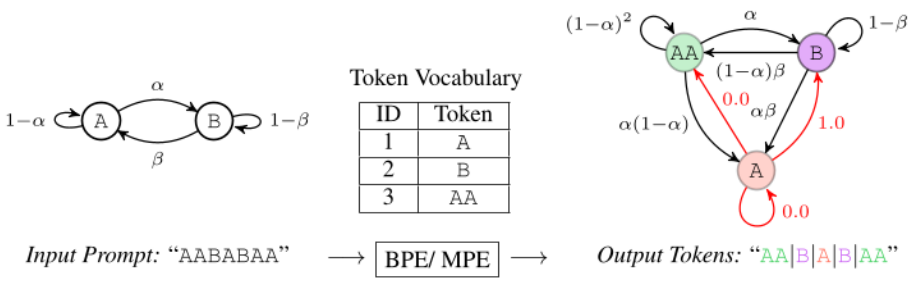


Figure 4: Tokenization bias on a 1st-order Markov chain. Given the context token "A", the model will never sample the next token as "A", rather than with probability $1 - \alpha$. In practice, this bias occurs when prompts end with tokens that are part of another token, a common issue in FIM tasks, leading to incorrect completions by the model.

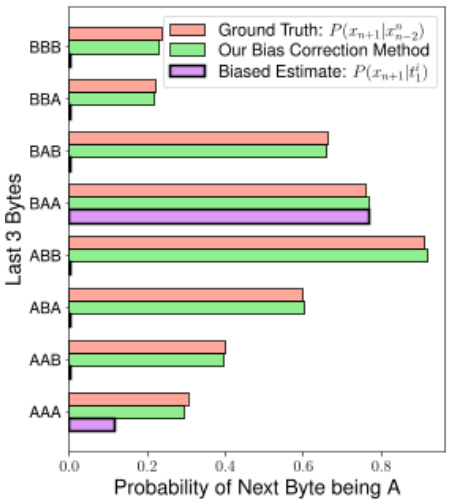


Figure 5: Tokenization bias on a 3rd-order Markov chain. Our byte-token conversion (bias correction) method in Section 3 accurately recovers $P(x_{n+1} \mid x_1^n)$ of the original chain.

4.3 Fill-in-the-middle code completion

Table 1 and Figure 6 report pass rates on the HumanEval Random Span benchmark with CodeLlama-7b; the figure shows how pass@10 varies with the sampling temperature.

SPM exposes tokenization bias. For the token-level model, switching from PSM to SPM drops the greedy pass rate from 64.9 to 45.0, and @10 from 84.0 to 66.5. The SPM prompt ends in the middle of a token, which puts the tokenized history out of distribution. Byte-level prediction recovers the SPM greedy score to 63.9 (+18.9 points over the token-level baseline) and @10 to 84.3, comparable to PSM.

Comparison with heuristics. Byte-level prediction is slightly higher than both heuristic fixes in SPM mode: 63.9 vs. 62.7 (healing) and 63.0 (alignment) greedy, and 84.3 vs. 83.8 and 82.9 at @10. The margins over the heuristics are small (0.9–1.2 points greedy, 0.5–1.4 points @10), and we have a single run per cell, so we regard this as consistent evidence that the exact method is at least as good as the heuristics rather than as a large improvement.

Gains in the benign case. Even for PSM, where tokenization issues are minimized, byte-level greedy generation edges out token-level (65.7 vs. 64.9), and @10 is essentially unchanged (84.1 vs. 84.0). The log also records qualitative failures: on prompts that end in the middle of a word (e.g. the variable names `delimeter`, `collatz` and `palindrome`), the token-level model fails to

Table 1: Pass rates (%) on the HumanEval Random Span infilling benchmark with CodeLlama-7b. PSM and SPM are the two infilling prompt templates; "@10" is pass@10. Healing and alignment are only applied in SPM mode (–: not applicable). Best per column in bold.

	PSM		SPM	
Method	Greedy	@10	Greedy	@10
Token-level	64.9	84.0	45.0	66.5
+ healing	–	–	62.7	83.8
+ alignment	–	–	63.0	82.9
Byte-level (ours)	**65.7**	**84.1**	**63.9**	**84.3**

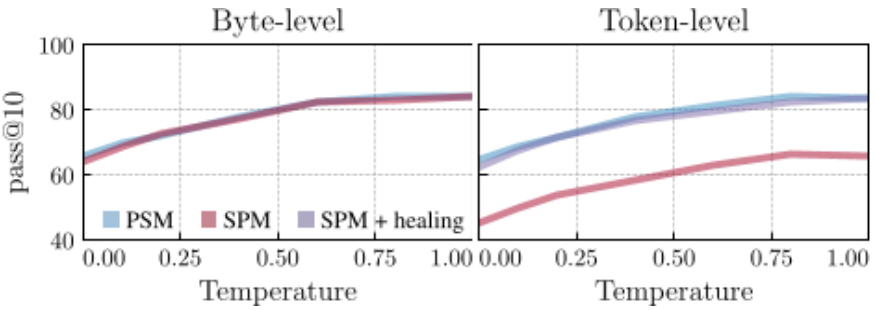


Figure 6: Pass@10 as a function of sampling temperature on the HumanEval Random Span benchmark (Bavarian et al., 2022) with CodeLlama-7b for infilling Python (the left panel of the supplied figure, a copy of Table 1, is omitted here). Across all settings, the byte-level predictions yield better completions. We evaluate the two standard infilling prompt templates of Prefix-Suffix-Middle (PSM) and Suffix-Prefix-Middle (SPM), and as baseline use token healing (Dagan et al., 2024) (alignment (Athiwaratkun et al., 2024) is compared in Table 1). The SPM template forces the model to make predictions from the middle of a token, which our byte-level predictions successfully handle.

complete the word because of the token-boundary mismatch, while the byte-level method identifies the correct completion.

Beam width. Table 2 shows that wider beams consistently help the byte-level model: pass rate rises from 65.7 to 67.7 in PSM mode and from 63.9 to 66.8 in SPM mode as the width grows from 1 to 4 (gains of 2.0 and 2.9 points). Width 1 corresponds to the greedy byte-level entries of Table 1.

4.4 Byte-level prediction and ensembling on general benchmarks

Is the conversion lossless in practice? In theory, byte-level conversion yields a statistically equivalent model, so benchmark scores should barely change when it is applied. Table 3 and Figure 7(a) show that this holds in most cells: for Yi-1.5-6B and Mistral-7B-v0.3 token- and byte-level scores differ by at most 0.3 points. Llama2-7B shows larger deviations on NQ (25.0 $\rightarrow$ 23.4) and GSM8K (15.1 $\rightarrow$ 12.5). On code generation (Table 4), byte-level HumanEval @1 is *lower* than token-level for all three single models (32.3 $\rightarrow$ 28.7, 35.8 $\rightarrow$ 33.5, 38.4 $\rightarrow$ 36.5), while MBPP is mixed (+1.8, $-$0.2, +0.8). These gaps are not predicted by the equivalence argument; our log does not isolate their cause, and we discuss possible reasons in Section 5.

Ensembling. Figure 7(b) summarizes the gains. Because byte distributions share one outcome space, we could aggregate models from three different families (Yi, Mistral, Llama) by simple averaging. The top-2 ensemble is higher than the best individual model on MMLU (65.4 vs. 63.4, Yi-1.5-6B), PIQA (80.7 vs. 80.3, Mistral-7B-v0.3), NQ (30.0 vs. 28.8), TriviaQA (64.2 vs. 63.6), and on HumanEval, where the ensemble of Yi-Coder-1.5B and CodeLlama-13b reaches 42.1, which is 3.7 points above the best individual token-level model (Yi-Coder-1.5B, 38.4; 5.6 above its byte-

Table 2: FIM pass rate (%) on HumanEval for byte-level predictions versus beam width; width 1 equals the byte-level greedy entries of Table 1. Best per column in bold.

Beam width	PSM	SPM
1	65.7	63.9
2	66.8	65.9
4	**67.7**	**66.8**

Table 3: Accuracy (%) of open-weight general-purpose LMs using standard token-level prediction (Token) and our byte-level prediction (Byte), and of top-2 ensembles (byte-level average of the two best-performing models, selected per benchmark). Voting was only reported for MMLU and PIQA (–: not reported). Best per benchmark in bold. Single evaluation per cell; no confidence intervals are available.

Model	Level	MMLU	PIQA	NQ	TriviaQA	GSM8K
Llama2-7B	Token	45.4	78.1	25.0	58.4	15.1
	Byte	45.7	78.2	23.4	58.1	12.5
Yi-1.5-6B	Token	63.4	78.5	22.8	53.7	61.3
	Byte	63.4	78.5	22.7	53.4	**61.5**
Mistral-7B-v0.3	Token	62.1	80.1	28.5	63.6	39.0
	Byte	62.1	80.3	28.8	63.5	39.0
Top-2 voting	Byte	62.1	80.1	–	–	–
Top-2 ensemble (ours)	Byte	**65.4**	**80.7**	**30.0**	**64.2**	55.8

level 36.5). It also exceeds the top-2 voting baseline on MMLU (65.4 vs. 62.1) and PIQA (80.7 vs. 80.1); voting was only evaluated for those two benchmarks. On MBPP the ensemble (53.6) matches the best single byte-level model and gives no gain. The gains on PIQA, NQ and TriviaQA are 0.4–1.2 points, within the range of the unexplained token/byte deviations above, so only the MMLU and HumanEval improvements are large relative to them. Even the HumanEval gain is based on a benchmark of only 164 problems, so 3.7 points amounts to a handful of problems and is suggestive rather than conclusive.

When ensembling fails. On GSM8K the ensemble (55.8) is worse than the best member (Yi-1.5-6B byte-level, 61.5). Here Yi-1.5-6B outperforms Mistral-7B-v0.3 by 22.5 points (61.5 vs. 39.0), and an equal-weight average lets the much weaker member degrade the stronger one. Ensembling with a uniform average is therefore beneficial only when the members have comparable capability on the task; weighting or selecting members is a natural remedy that we did not evaluate.

5 Conclusion

We identified tokenization bias as a mismatch between the next-token distribution of a tokenized LM and the byte-level distribution it implies, and showed how to remove it at inference time by marginalizing over valid cover encodings of the prompt, found by a backward search and aggregated by a Byte-Token Representation Lemma. On synthetic Markov chains the converted model recovers the true transition probabilities, on HumanEval Random Span infilling it removes most of the SPM degradation (45.0 $\rightarrow$ 63.9 greedy) and slightly exceeds token healing and alignment, and in byte space models with different vocabularies can be averaged, which improved over the best single model on MMLU, HumanEval, PIQA, NQ and TriviaQA.

Limitations. (i) Results are single-run numbers without confidence intervals, so small differences (e.g. over healing and alignment, or ensemble gains below about one point) should be read with care. (ii) Byte-level scores sometimes fall below token-level ones (Llama2-7B on NQ and GSM8K; all three code models on HumanEval), although the two should be equivalent in theory. We have not isolated the reason; candidates we did not test include model mass on invalid encodings that the lemma assumes away, and differences in how prompts and stopping are handled at the byte level.

Table 4: Pass@1 (%) of code LMs with token-level and byte-level prediction, and of the top-2 byte-level ensemble. Top-2 ensemble: byte-level average of the two best-performing models, selected per benchmark; on HumanEval the pair is Yi-Coder-1.5B and CodeLlama-13b. On MBPP the ensemble ties the best single model (Yi-Coder-1.5B, byte-level). Single evaluation per cell; no confidence intervals are available. Best per benchmark in bold.

Model	Level	HumanEval	MBPP
CodeLlama-7b	Token	32.3	40.6
	Byte	28.7	42.4
CodeLlama-13b	Token	35.8	47.6
	Byte	33.5	47.4
Yi-Coder-1.5B	Token	38.4	52.8
	Byte	36.5	**53.6**
Top-2 ensemble (ours)	Byte	**42.1**	**53.6**

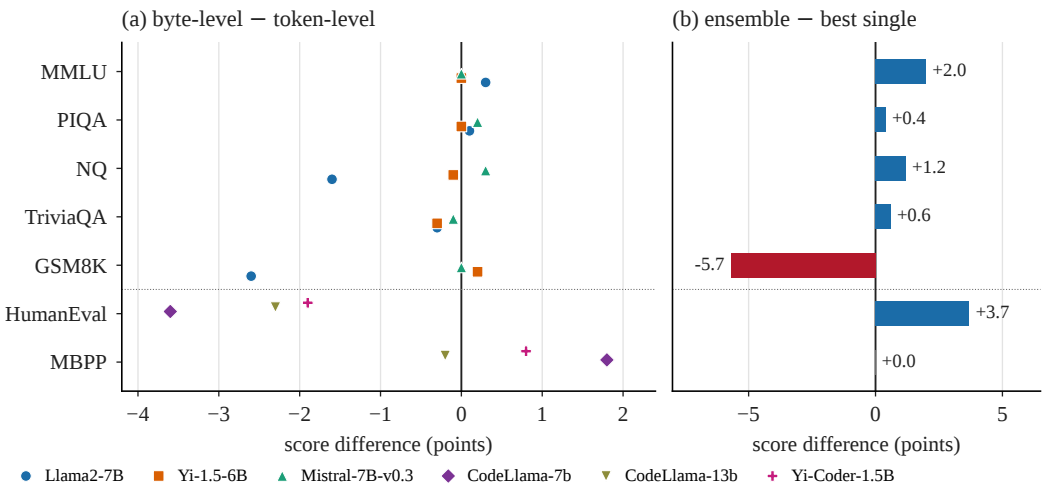


Figure 7: Differences in points. (a) Byte-level minus token-level score for each model and benchmark (Tables 3 and 4); (b) top-2 byte-level ensemble minus the best single model. GSM8K is the one benchmark where the ensemble is worse (-5.7); MBPP is a tie.

(iii) A uniform average fails when ensemble members differ greatly in ability (GSM8K). (iv) The method needs a deterministic tokenizer so that validity can be checked, and we report no runtime or memory measurements, so the overhead relative to token-level decoding is not quantified here. (v) We evaluated only Transformer LMs and a small set of benchmarks.

Future work. Natural directions include weighted or learned combinations for ensembles of unequal members, mixtures of experts over heterogeneous vocabularies, and a measured analysis of the computational cost of cover-encoding search for other tokenizers.

REFERENCES

Ben Athiwaratkun, Shiqi Wang, Mingyue Shang, et al. Token alignment via character matching for subword completion. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Guiding LLMs the right way: Fast, non-invasive constrained generation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Kris Cao and Laura Rimell. You should evaluate your language model on marginal likelihood over tokenisations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. Getting the most out of your tokenizer for pre-training and domain adaptation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Daniel Fried, Armen Aghajanyan, Jessy Lin, et al. InCoder: A generative model for code infilling and synthesis. In *International Conference on Learning Representations (ICLR)*, 2023.

Philip Gage. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38, 1994.

Juan Luis Gastaldi, John Terilla, Luca Malagutti, et al. The foundations of tokenization: Statistical and computational concerns. *arXiv preprint arXiv:2407.11606*, 2024.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023a.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023b.

Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7, 2019.

Sander Land and Max Bartolo. Fishing for magikarp: Automatically detecting under-trained tokens in large language models. *arXiv preprint arXiv:2405.05417*, 2024.

Raymond Li, Loubna Ben Allal, Yangtian Zi, et al. StarCoder: May the source be with you! *Transactions on Machine Learning Research*, 2023.

Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, et al. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. *arXiv preprint arXiv:2112.10508*, 2021.

Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.

Nived Rajaraman, Jiantao Jiao, and Kannan Ramchandran. Toward a theory of tokenization in LLMs. *arXiv preprint arXiv:2404.08335*, 2024.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.

Aaditya K. Singh and DJ Strouse. Tokenization counts: The impact of tokenization on arithmetic in frontier LLMs. *arXiv preprint arXiv:2402.14903*, 2024.

Kevin Slagle. SpaceByte: Towards deleting tokenization from large language modeling. *arXiv preprint arXiv:2404.14408*, 2024.

Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Junxiong Wang, Tushaar Gangavarapu, Jing Nathan Yan, and Alexander M. Rush. MambaByte: Token-free selective state space model. *arXiv preprint arXiv:2401.13660*, 2024.

Yangyifan Xu, Jinliang Lu, and Jiajun Zhang. Bridging the gap between different vocabularies for LLM ensemble. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10, 2022.

Alex Young, Bei Chen, Chao Li, et al. Yi: Open foundation models by 01.AI. *arXiv preprint arXiv:2403.04652*, 2024.

Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Yao-Ching Yu, Chun-Chih Kuo, Ziqi Ye, et al. Breaking the ceiling of the LLM community by treating token generation as a classification for ensembling. *arXiv preprint arXiv:2406.12585*, 2024.

Vilém Zouhar, Clara Meister, Juan Luis Gastaldi, Li Du, Tim Vieira, Mrinmaya Sachan, and Ryan Cotterell. A formal perspective on byte-pair encoding. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.

A Details of the Byte-Token Representation Lemma

We spell out the argument behind Lemma 1. Let T be the (random) token sequence generated by the tokenized LM, and assume its support consists of valid encodings, i.e. P is concentrated on $\{\mathrm{encode}(x)\}$. Let $X = \mathrm{decode}(T)$ be the byte string it represents. Then $P(X \text{ has prefix } x_1^n) = P(x_1^n)$ by definition of the byte-level process induced by the model. For each realization t with this property, let k be the smallest index such that $\mathrm{decode}(t_1^k)$ has at least n bytes. Then t_1^k has x_1^n as a byte prefix, and its last token is the first to reach position n, which is the defining property of $\mathrm{cover}(x_1^n)$ (Definition 4). Conversely, each element of $\mathrm{cover}(x_1^n)$ is the shortest covering prefix of the valid sequences extending it. The events $\{T \text{ extends } t_1^k\}$ for different $t_1^k \in \mathrm{cover}(x_1^n)$ are therefore disjoint and their union is the event $\{X \text{ has prefix } x_1^n\}$, and $P(T \text{ extends } t_1^k) = P(t_1^k)$ by the chain rule, which gives Eq. equation 1. We note that this argument relies on the model assigning no mass to invalid sequences, which holds only approximately for trained models. The algorithm therefore *checks* validity explicitly (Section 3.3) and rejects invalid candidates.

B Pseudocode for the reverse search

Algorithm 1 Cover encodings by backward tokenization

Require: byte string x_1^n, vocabulary $\mathcal{V}$, deterministic tokenizer encode

$\mathcal{C} \leftarrow \emptyset$
for $i = 0, 1, 2, \ldots$ **do**
  $\mathcal{T}_i \leftarrow \{t \in \mathcal{V} :\ t \text{ starts with } x_{n-i}^n\}$
  if $\mathcal{T}_i = \emptyset$ **then break** ▷ no token is long enough
  end if
  for $t \in \mathcal{T}_i$ **do**
    $t_1^k \leftarrow (\mathrm{encode}(x_1^{n-i-1}),\ t)$
    if $\mathrm{encode}(\mathrm{decode}(t_1^k)) = t_1^k$ **then**
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{t_1^k\}$
    end if
  end for
end for
return $\mathcal{C}$

C Notes on the reported results

All numbers in Section 4 are taken verbatim from the experimental log; the percentages in the text are differences in points (e.g. $63.9 - 45.0 = 18.9$). Our log names the code models in Table 4 "CodeLlama2-7b" and "CodeLlama2-13b"; we refer to them as CodeLlama-7b and CodeLlama-13b, consistent with Table 1. The qualitative failure cases for prompts ending mid-word (`delimeter`, `collatz`, `palindrome`) are described in the log without per-example probabilities, so we do not tabulate them.